

E. 你可以幫我吹口哨嗎?

Problem ID: Can_You_Blow_My_Whistle

Time Limit: 1s

Memory Limit: 512MiB



Figure 1: Can You Blow My Whistle Baby

本題採用函式互動形式。

國中營比賽當天，中午吃飯的時候，有 n 個小學弟排成一列準備領便當。

小學弟們身上穿著 1 到 n 號的衣服，且每個號碼恰好出現一次。

目前他們站在位置 1 到 n 上，位於位置 i 的學弟是 a_i 。

身為大學長的你，希望他們能依照號碼 1 到 n 的順序從小到大排好（也就是位置 i 要站的是編號 i 的學弟）。

為了節省時間，你決定吹哨子發號施令。每次吹哨，你可以同時指定任意多組學弟互換位置，但為了避免碰撞，每個學弟在同一次吹哨中，最多只能參與一組交換。

身為口哨先輩的你，請安排交換方式，用盡可能少的哨音讓所有學弟排好。

— 函式介面 —

你只需要提交實作 `solve` 函式的程式碼，不可以實作 `main` 函式。評測系統會將你的程式與 `grader` 一起編譯。

你的程式應包含以下標頭檔：

```
#include "Can_You_Blow_My_Whistle.h"
```

評測系統會呼叫：

- `void solve(int n, std::vector<int> a);`
 - a 的長度為 n ，且是 1 到 n 的一個排列。
 - 對於 $0 \leq i < n$ ， $a[i]$ 表示一開始站在位置 $i + 1$ 的學弟編號。
 - 你需要在此函式中呼叫下列函式安排交換；不需要回傳任何值。

你可以呼叫：

- `void swap_student(int u, int v);`
 - $1 \leq u, v \leq n$ 且 $u \neq v$ 。
 - 將「交換位置 u 與位置 v 」加入目前這一輪，但此時還不會立刻交換。
 - 同一輪中，每個位置最多只能出現在一次 `swap_student` 呼叫中，否則會得到 **Wrong Answer(1)**。
- `void blow_whistle();`
 - 同時執行目前這一輪加入的所有交換，接著開始新的一輪。
 - 即使這一輪沒有加入任何交換，呼叫此函式仍會計入一次哨音。

傳入的 a 是普通的 `std::vector<int>`；呼叫上述函式不會自動修改你程式中的 a 。
若 `solve` 結束時仍有尚未經過 `blow_whistle` 執行的交換，這些交換不會生效。

— 輸入限制 —

- $1 \leq n \leq 10^5$
- $1 \leq a_i \leq n$
- a 是一個排列

— 給分方式 —

假如你呼叫 `blow_whistle` 函式的次數超過了 $2n$ 次，則系統會中斷，並給出 **Wrong Answer(2)**。

當你的 `solve` 函式結束時，系統會檢查學弟們是否已經正確排序。若最終沒有完成排序，該筆測資的得分倍率為 0。

若成功完成排序，令該筆測資理論上最少需要的哨音數為 opt ，你呼叫 `blow_whistle` 的總次數為 A ，該筆測資的得分倍率 T 為：

- 若 $A = opt$ ，則 $T = 1$ 。
- 若 $A > opt$ ，則 $T = \max(0, 0.5 - 0.02(A - opt))$ 。

也就是說，只要哨音數不是最佳解，該筆測資的倍率最多為 0.5，且每多吹一次會再扣除 0.02，直到降為 0。

對於滿分為 P 的子任務，令 $T_{\min}$ 為你在該子任務所有測資中得到的最小倍率，則此子任務的得分為 $P \times T_{\min}$ 。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	34	$1 \leq n \leq 2000$
3	15	保證 $opt = 1$
4	51	無額外限制

— 範例 —

評測系統會以 `solve(5, {2, 1, 3, 5, 4})` 呼叫你的函式。

例如可以依序呼叫：

```
swap_student(1, 2);  
swap_student(4, 5);  
blow_whistle();
```

前兩個呼叫將兩組交換加入同一輪；呼叫 `blow_whistle` 後，兩組交換同時執行，排列變成 `[1, 2, 3, 4, 5]`。此例最少需要一次哨音。